
Iteración 0

Arranque del desarrollo e incremento 0.1

Una aplicación web que genera aplicaciones web

Documento **SLCP-DOC-003** – Versión **1.1**

6 de agosto de 2026

Este documento registra el arranque efectivo del desarrollo. Recoge la decisión de que SLCP es una aplicación web que genera aplicaciones web y sus consecuencias sobre los requisitos vigentes; documenta el incremento 0.1, ya construido y verificado, con su evidencia de ejecución; deja constancia de la enmienda al oráculo detectada durante la verificación, que queda pendiente de ratificación; y fija la secuencia de los incrementos siguientes junto con lo que permanece bloqueado.

Cambios de la versión 1.1: el incremento 0.1 queda consolidado tras la ratificación de la enmienda y la incorporación del validador de nombres; se actualiza el inventario de lo que falta para continuar.

Índice

1	Decisión registrada: naturaleza web en ambos extremos	2
2	Condiciones de verificación del entorno actual	2
3	Incremento 0.1: transformación de nomenclatura	2
3.1	Qué es	3
3.2	Requisitos realizados	3
3.3	Decisiones de diseño adoptadas	3
3.4	Evidencia de verificación	3
4	Enmienda A-001 al oráculo	4
5	Secuencia de incrementos siguientes	4
6	Qué falta para continuar	5
6.1	Respuesta directa: ¿hace falta el documento de requisitos ahora?	5
6.2	El límite que no depende de ninguna decisión	5

1 Decisión registrada: naturaleza web en ambos extremos

SLCP es una aplicación web, y las aplicaciones que genera son también aplicaciones web. La coincidencia no es casual ni irrelevante: significa que el nivel [N1] y el nivel [N3] comparten arquitectura de referencia, y que la plataforma puede emplear para sí misma las mismas plantillas que produce para otros, cuando llegue el momento de la autoaplicación descrita en SLCP-DOC-000.

Esta decisión cierra D-05 de SLCP-DOC-002 en su opción (b), servidor autoalojado con varios usuarios y un solo inquilino institucional, y precisa el alcance de los requisitos TGT: los destinos obligatorios de TGT-01 y TGT-02 deben entenderse en el contexto de aplicaciones web, es decir, servicios HTTP con persistencia relacional e interfaz de navegador, no aplicaciones de escritorio ni procesos por lotes.

Nivel	Naturaleza	Consecuencia
[N1]	Aplicación web: interfaz de navegador, API HTTP, persistencia relacional	La plataforma es accesible sin instalación en el puesto; la identidad y la aprobación de TRC-22 se resuelven en sesión autenticada
[N3]	Aplicaciones web generadas	Las plantillas producen servicio HTTP, esquema relacional, contrato OpenAPI e interfaz de navegador

2 Condiciones de verificación del entorno actual

Antes de escribir código se comprobó qué puede verificarse de forma efectiva en el entorno de trabajo disponible. El resultado condiciona el orden de los incrementos y se documenta aquí porque no hacerlo llevaría a entregar código no ejecutado.

Capacidad	Estado	Consecuencia
JDK 21 con compilador	Disponible	Java sin dependencias externas es compilable y ejecutable
Repositorio de artefactos Java	No accesible	No pueden construirse aquí módulos con dependencias de terceros
Motor relacional	No disponible	El esquema no puede ejecutarse contra un motor real todavía
Node y su registro	Disponible	La interfaz de navegador sí podrá construirse y probarse

De ahí la elección del primer incremento: se ha seleccionado deliberadamente el módulo que puede verificarse por completo con las capacidades existentes y del que, además, dependen todos los generadores posteriores. Los incrementos que exigen persistencia o marco de servicio quedan para cuando el repositorio del proyecto esté operativo en su entorno.

3 Incremento 0.1: transformación de nomenclatura

3.1 Qué es

El módulo `core/naming` implementa la función única y determinista que convierte un término canónico del glosario en el identificador correspondiente a cada destino. Es el cimiento de toda la generación: cada plantilla de cada lenguaje invoca esta función en lugar de resolver la nomenclatura por su cuenta, que es lo que exige NAM-04 y lo que evita que dos plantillas produzcan nombres distintos para el mismo concepto.

3.2 Requisitos realizados

Realiza NAM-01 en su parte de transliteración sin traducción, NAM-04, NAM-05, NAM-06 y NAM-07, y aplica a sí mismo el principio de TGT-08.

3.3 Decisiones de diseño adoptadas

La pluralización no se infiere. Deducir el plural mediante reglas produce resultados incorrectos en cuanto el idioma del glosario no es inglés o el término es irregular, y los produce en silencio. El módulo exige la forma plural del glosario y aborta si falta, conforme a NAM-07.

Los términos de dominio conservan su idioma. Un término como *Matrícula* produce el tipo *Matricula* y la tabla *matriculas*: se transliteran los diacríticos porque ningún destino los admite sin entrecomillar, pero no se traduce nada. Es la aplicación directa de la corrección de NAM-01 registrada en SLCP-DOC-000.

La colisión aborta. Ante una palabra reservada el módulo lanza un error con código estable en lugar de entrecomillar el identificador. Entrecomillar funcionaría en el momento y rompería la portabilidad después, porque PostgreSQL pliega a minúsculas y Oracle a mayúsculas los identificadores sin entrecomillar.

La abreviatura es determinista y libre de colisiones. Al exceder el límite se conserva un prefijo legible y se anexa una firma derivada del identificador completo. Su reversibilidad es por consulta al glosario, no por reconstrucción, y así queda anotado como limitación.

3.4 Evidencia de verificación

```
javac -encoding UTF-8 -Xlint:all -d out src/org/slcp/core/naming/*.java
-> codigo de salida 0, sin advertencias

java -cp out org.slcp.core.naming.VectorRunner naming-vectors.tsv
-> Vectores superados: 55
    Vectores fallidos: 0
    RESULTADO: VERDE
```

El oráculo cubre el vocabulario canónico del metamodelo, términos compuestos, indiferencia a la forma de entrada, términos de dominio en español con diacríticos, ausencia de plural, colisión con palabra reservada en los cuatro tipos de destino, y la regla de longitud con su valor esperado calculado de forma independiente de la implementación.

4 Enmienda A-001 al oráculo

La primera ejecución arrojó un fallo. Su análisis mostró que el defecto estaba en el oráculo y no en la implementación: el oráculo exigía sensibilidad a mayúsculas en Java y la negaba en C#, lo que es internamente contradictorio. El criterio correcto no es uniforme sino que depende de las reglas reales de cada destino: Java y C# tratan sus palabras clave de forma sensible a mayúsculas, PHP de forma insensible, y los dialectos SQL pliegan los identificadores sin entrecomillar.

La enmienda queda registrada en `oracle-amendments.md` con su análisis y su evidencia, la versión previa del oráculo se conserva como `naming-vectors.tsv.v0`, y se han añadido cuatro vectores que fijan de forma explícita la sensibilidad de cada destino para que la contradicción no pueda reaparecer.

Acción requerida. Conforme a TGT-08, la enmienda de un oráculo exige aprobación humana independiente. El incremento 0.1 no puede alcanzar línea base hasta que usted ratifique la enmienda A-001. Hasta entonces su estado es provisional.

Conviene subrayar lo ocurrido, porque es la primera prueba empírica de que el proceso adoptado funciona: el oráculo detectó una contradicción que la implementación por sí sola habría ocultado, y el procedimiento obligó a registrarla en lugar de corregirla en silencio.

5 Secuencia de incrementos siguientes

Tabla 1: Plan de la Iteración 0

Incr.	Contenido	Condición para abordarlo
0.1	Transformación de nomenclatura y validación de nombres	Consolidado: 76 vectores en verde, enmienda ratificada
0.2	Metamodelo del ciclo de vida como esquema formal, con su glosario y el registro de abreviaturas	Ninguna; puede abordarse de inmediato
0.3	Almacén de solo anexo y esquema relacional, con la prueba negativa de TRC-24	Requiere motor relacional en su entorno
0.4	Importador y exportador ReqIF con la prueba de ida y vuelta de INT-09	Requiere el caso semilla C-01
0.5	Esqueleto de la aplicación web: API y navegación mínima	Requiere D-01, D-04 y repositorio operativo

El criterio de salida de la Iteración 0 sigue siendo el fijado en SLCP-DOC-002: que el caso semilla pueda importarse, persistirse con identificadores conformes, exportarse y reimportarse sin diferencias semánticas.

6 Qué falta para continuar

Conviene separar tres cosas que suelen confundirse al preguntar qué hace falta: las decisiones que solo la parte interesada puede tomar, los insumos que hay que aportar, y las condiciones materiales del entorno de trabajo. Las tres son necesarias, pero no bloquean lo mismo.

Tabla 2: Inventario de lo pendiente

Qué falta	Qué es	Qué bloquea exactamente
Licencia del código	Una elección entre dos	La primera dependencia externa que se incorpore
Primer destino de generación	Confirmar Java, Spring Boot y PostgreSQL	La primera plantilla de generación
Documento de requisitos real	Una especificación auténtica de un proyecto cualquiera	El diseño del analizador y toda comprobación de extremo a extremo
Repositorio operativo	Una dirección donde depositar el trabajo	La continuidad: hoy el código vive en un entorno provisional
Entorno con acceso a repositorios de artefactos y motor relacional	Una máquina donde funcionen las herramientas de construcción	Todo lo que use marcos de trabajo o base de datos

6.1 Respuesta directa: ¿hace falta el documento de requisitos ahora?

No para lo inmediato, y sí para lo que viene después.

El incremento siguiente, que es el metamodelo del ciclo de vida con su glosario y su registro de abreviaturas, no depende de él: se construye con las mismas herramientas con que se construyó el primero y puede abordarse sin esperar nada. El documento de requisitos hace falta a partir del momento en que haya que comprobar que algo funciona de extremo a extremo, porque hasta entonces no hay nada real contra lo que contrastar.

6.2 El límite que no depende de ninguna decisión

Hay una restricción que conviene enunciar con claridad porque ninguna respuesta suya la levanta. El entorno de trabajo actual no puede descargar artefactos de terceros ni dispone de motor relacional, de modo que aquí no es posible construir ni la interfaz ni el servicio de la aplicación web. Lo que sí puede producirse es todo aquello que no dependa de artefactos externos: el metamodelo formal, los esquemas, las reglas, los oráculos y los módulos de núcleo sin dependencias, tal como se hizo con el primero.

Dicho de otro modo: la aplicación web se construirá en su entorno, no en éste. Lo que se produzca aquí seguirá siendo material verificado que se incorpora a aquel.

Referencias